43-search-implementation

43. Search Implementation

Level: Intermediate
Jam: 8 (4 sesi)
Prasyarat: Paham Node.js, Express, PostgreSQL dasar, REST API
Output: Search feature with autocomplete, full-text search, typo tolerance

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Implementasi full-text search di PostgreSQL pake tsvector/tsquery + GIN index
- Setup Meilisearch sebagai dedicated search engine — index, filter, faceted search
- Bangun client-side fuzzy search pake Fuse.js & lunr.js
- Desain search architecture: hybrid search, data sync, search analytics
- Handle typo tolerance, autocomplete, search result highlighting

Materi

Sesi	Topik	File
1	PostgreSQL Full-Text Search — tsvector/tsquery, GIN index, ranking, highlight, bahasa	01-postgres-fts.md

Sesi	Topik	File
2	Meilisearch — setup, index, filterable/faceted search, typo tolerance, ranking rules	02-meilisearch.md
3	Client-Side Search — Fuse.js, lunr.js, debounce, autocomplete, lazy load index	03-client-side-search.md
4	Search Architecture — single vs dedicated service, data sync (webhook/cron/CDC), hybrid, analytics, multi-language	04-search-architecture.md

Output Akhir Modul

> **Search Feature** — aplikasi dengan full-text search di PostgreSQL untuk query akurat, Meilisearch untuk fast typo-tolerant search, dan Fuse.js untuk client-side fuzzy fallback. Dilengkapi autocomplete dropdown, search result highlighting, dan search analytics dashboard.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Explain the difference between LIKE '%keyword%' and PostgreSQL full-text search using tsvector — include performance and feature comparison"
- "Generate SQL to create a GIN index on a products table with name and description columns for full-text search in Indonesian language config"
- "Compare Meilisearch vs Elasticsearch vs PostgreSQL FTS for a startup building a search feature with < 100k documents — include cost, complexity, and performance considerations"
- "Write a Fuse.js configuration that searches across title, description, and tags with typo tolerance of 2 and returns top 10 results sorted by score"
- "Design a search sync architecture: a Node.js app writes to PostgreSQL, data needs to be indexed in Meilisearch within 30 sec-

onds. Compare webhook, cron-based, and CDC approaches”
- “Generate code for a debounced search input component with autocomplete dropdown that calls an API endpoint after 300ms idle”
- “How to handle multi-language search where some documents are in English and others in Indonesian? Compare per-language index vs unified approach”

01. PostgreSQL Full-Text Search

Kenapa LIKE %keyword% Gak Cukup?

Search pake `ILIKE '%cari%'` punya masalah besar:

- **Gak pake index** — query full table scan, lambat di tabel besar
- **Gak ngerti bahasa** — `'lari'` gak match `'berlari'` atau `'larilah'`
- **Gak bisa ranking** — mana hasil paling relevan? semua return
- **Gak handle typo** — `'lari'` vs `'lariii'`

```
-- ❌ Lambat & terbatas
SELECT * FROM articles WHERE title ILIKE '%indonesia%';

-- ✅ Cepet & pinter — full-text search
SELECT * FROM articles WHERE search_vector @@ to_tsquery('indonesia');
```

Tsvector & Tsquery

tsvector — Ubah teks jadi searchable tokens

`to_tsvector('english', text)` → parse teks, ubah ke lexemes (kata dasar), hapus stop words.

```
SELECT to_tsvector('english', 'The cats are running through the garden');
-- output: 'cat':2 'garden':6 'run':3
-- "the", "are", "through" ilang (stop words)
-- "cats" → "cat" (lemma)
-- "running" → "run" (lemma)
```

tsquery — Query format buat search

```
SELECT to_tsquery('english', 'cat & run');
-- output: 'cat' & 'run'

SELECT to_tsquery('english', 'cat | dog');
SELECT to_tsquery('english', '!bird');
```

```sql
SELECT to_tsquery('english', 'cucumber <-> sandwich');
-- <-> = phrase (distance 1)
```

Match operator `@@`

```sql
-- AND match
SELECT 'cat run garden'::tsvector @@ to_tsquery('cat & run'); -- true

-- OR match
SELECT 'cat dog'::tsvector @@ to_tsquery('cat | bird'); -- true

-- NOT match
SELECT 'cat dog'::tsvector @@ to_tsquery('cat & !bird'); -- true
```

Setup Full-Text Search di PostgreSQL

1. Tambah kolom tsvector

```sql
-- Tambah kolom search_vector
ALTER TABLE articles ADD COLUMN search_vector tsvector;

-- Isi dari kolom title + body
UPDATE articles
SET search_vector = to_tsvector('english', coalesce(title,'') || ' ' || coalesce
```

2. Auto-update pake trigger

```sql
CREATE FUNCTION articles_search_update() RETURNS trigger AS $$
BEGIN
  NEW.search_vector := to_tsvector('english', coalesce(NEW.title,'') || ' ' || c
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_articles_search
  BEFORE INSERT OR UPDATE ON articles
  FOR EACH ROW EXECUTE FUNCTION articles_search_update();
```

3. GIN Index — biar cepet

```sql
CREATE INDEX idx_articles_search ON articles USING GIN(search_vector);
```

Kenapa GIN (Generalized Inverted Index)? — nyimpen mapping dari token → row. Sama kayak index di buku: cari kata → langsung ke halaman.

4. Query search

```
SELECT id, title, body
FROM articles
WHERE search_vector @@ to_tsquery('english', 'indonesia & ekonomi')
ORDER BY ts_rank(search_vector, to_tsquery('english', 'indonesia & ekonomi')) DE
LIMIT 20;
```

Search Ranking — ts_rank & ts_rank_cd

ts_rank — frekuensi + kemunculan

```
SELECT
  id, title,
  ts_rank(search_vector, query) AS score
FROM articles, to_tsquery('english', 'indonesia & ekonomi') query
WHERE search_vector @@ query
ORDER BY score DESC;
```

Skor tinggi kalau keyword muncul berkali-kali. Tapi biasa bias ke dokumen panjang.

ts_rank_cd — coverage density

```
SELECT
  id, title,
  ts_rank_cd(search_vector, query) AS score
FROM articles, to_tsquery('english', 'indonesia & ekonomi') query
WHERE search_vector @@ query
ORDER BY score DESC;
```

ts_rank_cd bagi skor sama panjang dokumen — fair buat dokumen pendek vs panjang.

Weighted ranking — judul lebih penting

```
-- setweight: A=1.0, B=0.4, C=0.2, D=0.1
SELECT
  id, title,
  ts_rank(
    setweight(to_tsvector('english', coalesce(title,'')), 'A') ||
    setweight(to_tsvector('english', coalesce(body,'')), 'B'),
    query
  ) AS score
FROM articles, to_tsquery('english', 'indonesia') query
WHERE to_tsvector('english', coalesce(title,'') || ' ' || coalesce(body,'')) @@
ORDER BY score DESC;
```

Language Config

PostgreSQL support banyak language config buat stemming & stop words.

English

```
SELECT to_tsvector('english', 'running faster'); -- 'run' 'fast'
```

Simple (no stemming, no stop words)

```
SELECT to_tsvector('simple', 'Running running RUNNING'); -- 'running' 'running'
```

Berguna kalau mau exact match case-insensitive.

Indonesian

```
SELECT to_tsvector('indonesian', 'berlarian berlari'); -- 'lari' 'lari'
```

Daftar language bawaan: `english`, `indonesian`, `simple`, `dutch`, `french`, `german`, `spanish`, dll.

Cek yg tersedia:

```
SELECT cfgname FROM pg_ts_config;
```

Multi-language column

```
ALTER TABLE articles ADD COLUMN lang varchar(2) DEFAULT 'en';

CREATE FUNCTION articles_search_update() RETURNS trigger AS $$
DECLARE
  config regconfig := 'simple';
BEGIN
  config := CASE NEW.lang
    WHEN 'en' THEN 'english'::regconfig
    WHEN 'id' THEN 'indonesian'::regconfig
    ELSE 'simple'::regconfig
  END;
  NEW.search_vector := to_tsvector(config, coalesce(NEW.title,'') || ' ' || coal
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Query Syntax Variations

`plainto_tsquery` — plain text → tsquery (AND)

```
-- 'indonesia ekonomi' → 'indonesia' & 'ekonomi'
SELECT * FROM articles
WHERE search_vector @@ plainto_tsquery('english', 'indonesia ekonomi');
```

`phraseto_tsquery` — exact phrase

```
-- 'wisata alam' → 'wisata' <-> 'alam'
SELECT * FROM articles
WHERE search_vector @@ phraseto_tsquery('english', 'wisata alam');
```

`websearch_to_tsquery` — Google-like syntax

```
-- "wisata alam" = phrase, -jakarta = exclude, OR = or
SELECT * FROM articles
WHERE search_vector @@ websearch_to_tsquery('english', '"wisata alam" -jakarta O
```

Syntax: `"..."` phrase, `-` exclude, `OR` logical OR, sisanya AND.

Highlight Results — `ts_headline`

Tampilin snippet dengan keyword yang di-highlight.

```
SELECT
  id, title,
  ts_headline('english', body, query,
    'StartSel=<mark>, StopSel=</mark>,
     MaxWords=50, MinWords=20,
     HighlightAll=FALSE'
  ) AS snippet
FROM articles, to_tsquery('english', 'indonesia & ekonomi') query
WHERE search_vector @@ query;
```

Output: `"Perekonomian <mark>Indonesia</mark> tumbuh 5%..."`

Parameter `ts_headline`: - `StartSel`/`StopSel` — tag HTML buat highlight - `MaxWords`/`MinWords` — panjang snippet - `HighlightAll` — semua kata atau cuma sebagian - `FragmentDelimiter` — pemisah antar fragmen (default `...`)

Full Query Pattern

```
WITH query AS (
  SELECT plainto_tsquery('english', 'wisata alam indonesia') AS q
```

```
)
SELECT
  a.id, a.title, a.slug,
  ts_rank_cd(a.search_vector, q.q) AS score,
  ts_headline('english', a.body, q.q,
    'StartSel=<mark class="hl">, StopSel=</mark>, MaxWords=40, MinWords=15'
  ) AS snippet
FROM articles a, query q
WHERE a.search_vector @@ q.q
  AND a.published = true
ORDER BY score DESC
LIMIT 20;
```

Performance Tips

1. **Partial index** — cuma search di published articles

```
CREATE INDEX idx_articles_search_published
ON articles USING GIN(search_vector)
WHERE published = true;
```

2. **Combine dengan filter lain**

```
-- GIN + btree — PostgreSQL bisa combine di bitmap scan
CREATE INDEX idx_articles_category ON articles(category_id);
```

3. **Vacuum & analyze** — jaga index tetap optimal

```
ANALYZE articles;
```

4. **Limit hasil** — gak perlu return 1000 row
5. **Gunakan `websearch_to_tsquery`** — user-friendly, handle special chars otomatis

Latihan

Latihan 1: Setup Full-Text Search

Buat tabel `products` dengan kolom `name`, `description`, `category`. Tambah kolom `search_vector`, trigger auto-update, dan GIN index. Query cari produk berdasarkan kata kunci “kopi robusta”.

Latihan 2: Ranking & Weight

Buat query search produk dengan weighted ranking: judul (weight A), deskripsi (weight B), kategori (weight C). Tampilkan skor dan pastikan produk dengan judul match muncul di atas.

Latihan 3: Highlight & Snippet

Search artikel dengan keyword "berita politik terbaru". Tampilkan title, score, dan snippet dengan keyword di-highlight pake `<mark>` tag. Batasi snippet 30-50 kata.

Latihan 4: Multi-Language Search

Buat tabel `documents` dengan kolom `title`, `content`, `lang` (en/id). Konfigurasi trigger pake language config sesuai kolom `lang`. Coba search di kedua bahasa dan bandingkan hasil stemming-nya.

02. Meilisearch

Apa itu Meilisearch?

Meilisearch adalah search engine dedicated yang:

- **Typo-tolerant** — otomatis handle typo sampai configurable distance
- **Instant search** — response < 50ms, cocok buat autocomplete
- **Filter & faceted** — filter + aggregasi data di hasil search
- **Ranking rules** — bisa atur prioritas relevansi
- **Auto-complete** — built-in, gak perlu setup sendiri
- **REST API** — gak perlu SDK khusus, pake HTTP biasa

Perbandingan:

Fitur	PostgreSQL FTS	Meilisearch	Elasticsearch
Setup	Built-in	Docker/paket	Docker/JVM
Typo tolerance	□ Manual	□ Built-in	□ Custom
Speed (1M docs)	~50-200ms	~10-50ms	~50-200ms
Complex query syntax	□ TSQuery	□ REST API	□ DSL JSON
Devops overhead	None	Minimal	Tinggi
Use case	Secondary search	Primary search	Enterprise search

Setup Meilisearch

Docker (recommended)

```
docker run -d --name meilisearch \
  -p 7700:7700 \
  -e MEILI_MASTER_KEY=master-key-rahasia \
  -v $(pwd)/meili_data:/meili_data \
  getmeili/meilisearch:v1.10
```

Default: `http://localhost:7700`. Master key wajib di production.

Local binary (Linux)

```
curl -L https://github.com/meilisearch/meilisearch/releases/latest/download/meil
chmod +x meilisearch
./meilisearch --master-key=master-key-rahasia
```

Node.js SDK

```
npm install meilisearch
```

```
import { MeiliSearch } from 'meilisearch';

const client = new MeiliSearch({
  host: 'http://localhost:7700',
  apiKey: 'master-key-rahasia'
});
```

Index & Documents

Buat index

```
curl -X POST 'http://localhost:7700/indexes' \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer master-key-rahasia' \
  --data-binary '{ "uid": "products", "primaryKey": "id" }'
```

`uid` = nama index, `primaryKey` = field unique identifier.

Add documents

```
const documents = [
  {
    id: 1,
    name: "Kopi Robusta Aceh",
    description: "Kopi robusta dari dataran tinggi Aceh dengan rasa earthy",
    category: "Minuman",
    price: 45000,
    stock: 100,
    tags: ["kopi", "robusta", "aceh"]
  },
  {
    id: 2,
    name: "Teh Hijau Jawa Barat",
    description: "Teh hijau premium dari perkebunan Gambung",
```

```
    category: "Minuman",
    price: 35000,
    stock: 200,
    tags: ["teh", "hijau", "jawa"]
  }
];

const response = await client.index('products').addDocuments(documents);
// { taskUid: 1 }
```

Cek status indexing

```
const task = await client.getTask(1);
// { status: "succeeded", type: "documentAdditionOrUpdate", ... }
```

Meilisearch async — addDocuments return taskUid, cek status pake getTask.

Searchable Attributes

Default — semua attribute bisa di-search

Pengaturan:

```
searchableAttributes = ["*"]
```

Custom — tentuin field mana yg di-search

```
await client.index('products').updateSearchableAttributes([
  'name',
  'description',
  'tags'
]);
```

Prioritas sesuai urutan array. Name paling penting, terus description, terus tags.

```
// Cari keyword "kopi" — name > description > tags
const results = await client.index('products').search('kopi');
```

Filterable & Faceted Search

Filterable attributes

Set attribute mana yg bisa di-filter:

```
await client.index('products').updateFilterableAttributes([
  'category',
```

```
  'price',
  'stock',
  'tags'
]);
```

Filter syntax

```
// Filter by kategori
const results = await client.index('products').search('kopi', {
  filter: 'category = "Minuman"'
});

// Range filter
const results = await client.index('products').search('kopi', {
  filter: 'price >= 40000 AND price <= 100000'
});

// Array filter
const results = await client.index('products').search('teh', {
  filter: 'tags IN [teh, hijau]'
});
```

Filter bisa numeric, string, boolean, dan array.

Faceted search — aggregasi

```
await client.index('products').updateFilterableAttributes([
  'category',
  'price'
]);

const results = await client.index('products').search('', {
  facets: ['category', 'price']
});

console.log(results.facetDistribution);
// {
//   category: { Minuman: 25, Makanan: 15 },
//   price: { '0-50000': 20, '50000-100000': 15, ... }
// }
```

Cocok buat e-commerce filter sidebar — kategori, harga, rating.

Numeric filter

```
// Numeric comparison
filter: 'price >= 10000 AND price <= 50000'

// OR
filter: 'category = "Minuman" OR category = "Snack"'

// Nested
filter: '(price >= 10000 AND price <= 50000) OR stock > 0'
```

Pagination & Typo Tolerance

Pagination

```
const results = await client.index('products').search('kopi', {
  limit: 20,   // default 20, max 1000
  offset: 0    // halaman ke-1
});

// Hitung halaman
const totalPages = Math.ceil(results.totalHits / results.limit);
const currentPage = Math.floor(results.offset / results.limit) + 1;
```

Typo tolerance — configurable

Meilisearch otomatis tolerate typo. Default rules:

Panjang kata	Max typo
1-4 char	1 typo
5-8 char	2 typo
9+ char	2 typo

Override:

```
await client.index('products').updateTypoTolerance({
  enabled: true,
  minWordSizeForTypos: {
    oneTypo: 4,    // kata 4+ char: 1 typo
    twoTypos: 8    // kata 8+ char: 2 typo
  },
  disableOnWords: [],
  disableOnAttributes: []
});
```

Matikan typo buat field tertentu (misal kode produk):

```
await client.index('products').updateTypoTolerance({
  enabled: true,
  disableOnAttributes: ['sku', 'barcode']
});
```

Cara kerja typo tolerance

Meilisearch pake Levenshtein distance algorithm:

- `kpi` → match `kopi` (distance 1 — k vs o)
- `robusta` → match `robusta` (distance 0 — exact)
- `robusta` → match `robusta` (distance 1 — transposisi)

Search Relevance Tuning

Ranking rules — urutan prioritas relevansi

Default ranking rules:

```
["words", "typo", "proximity", "attribute", "sort", "exactness"]
```

Rule	Fungsi
`words`	Makin banyak kata match → makin tinggi
`typo`	Makin sedikit typo → makin tinggi
`proximity`	Kata berdekatan → makin tinggi
`attribute`	Match di field prioritas (searchableAttributes)
`sort`	Sort manual (kalau ada)
`exactness`	Exact phrase match → tertinggi

Custom ranking:

```
await client.index('products').updateRankingRules([
  'words',
  'typo',
  'proximity',
  'attribute',
  'sort',
  'exactness',
  'price:asc'     // ascending price
]);
```

Custom ranking attributes

Buat sorting pake data sendiri:

```
// 1. Set custom ranking
await client.index('products').updateRankingRules([
  'words',
  'typo',
  'proximity',
  'attribute',
  'sort',
  'exactness',
  'popularity:desc'  // produk populer dulu
]);

// 2. Pastiin field ada di dokumen
const documents = [
  {
    id: 1,
    name: "Kopi Robusta Aceh",
    popularity: 95,   // 0-100
    // ...
  }
];
```

Sort parameter di query

```
// Sort asc
await client.index('products').search('kopi', {
  sort: ['price:asc']
});

// Sort desc
await client.index('products').search('kopi', {
  sort: ['popularity:desc']
});

// Multi-sort
await client.index('products').search('kopi', {
  sort: ['popularity:desc', 'price:asc']
});
```

Full Integration Example — Node.js

```
import { MeiliSearch } from 'meilisearch';

const client = new MeiliSearch({
  host: 'http://localhost:7700',
  apiKey: process.env.MEILI_MASTER_KEY
```

```
});

const INDEX_NAME = 'products';

// 1. Setup index
async function setupIndex() {
  await client.createIndex(INDEX_NAME, { primaryKey: 'id' });

  await client.index(INDEX_NAME).updateSearchableAttributes([
    'name', 'description', 'tags'
  ]);

  await client.index(INDEX_NAME).updateFilterableAttributes([
    'category', 'price', 'stock', 'tags'
  ]);

  await client.index(INDEX_NAME).updateRankingRules([
    'words', 'typo', 'proximity', 'attribute',
    'sort', 'exactness', 'popularity:desc'
  ]);
}

// 2. Index products from DB
async function indexProducts(products) {
  const { taskUid } = await client.index(INDEX_NAME).addDocuments(products);

  // Wait for indexing
  let task = await client.getTask(taskUid);
  while (task.status === 'processing' || task.status === 'enqueued') {
    await new Promise(r => setTimeout(r, 100));
    task = await client.getTask(taskUid);
  }

  if (task.status === 'failed') {
    throw new Error(`Indexing failed: ${task.error.message}`);
  }

  return task;
}

// 3. Search API handler
async function searchProducts(query, filters = {}) {
  const searchParams = {
    limit: filters.limit || 20,
    offset: filters.offset || 0,
    filter: buildFilter(filters),
```

```
    sort: filters.sort || ['popularity:desc'],
    facets: ['category']
  };

  const results = await client.index(INDEX_NAME).search(query, searchParams);

  return {
    hits: results.hits,
    totalHits: results.totalHits,
    facetDistribution: results.facetDistribution,
    processingTimeMs: results.processingTimeMs
  };
}

function buildFilter(filters) {
  const conditions = [];

  if (filters.category) {
    conditions.push(`category = "${filters.category}"`);
  }
  if (filters.minPrice) {
    conditions.push(`price >= ${filters.minPrice}`);
  }
  if (filters.maxPrice) {
    conditions.push(`price <= ${filters.maxPrice}`);
  }
  if (filters.inStock) {
    conditions.push(`stock > 0`);
  }

  return conditions.length > 0 ? conditions.join(' AND ') : undefined;
}

export { setupIndex, indexProducts, searchProducts };
```

Production Checklist

- **Set master key** — gak ada anonymous access
- **Dump & backup** — `curl .../dump` buat backup index
- **Snapshots** — enable auto-snapshot `--snapshot-interval-sec=3600`
- **Resource** — Meilisearch butuh ~500MB RAM per 1M docs
- **API rate limit** — Meilisearch gak punya built-in, pake reverse proxy
- **Environments** — separate index for dev/staging/production

```
docker run -d --name meilisearch \
  -p 7700:7700 \
  -e MEILI_MASTER_KEY=master-key-prod \
  -e MEILI_ENV=production \
  -e MEILI_DUMP_DIR=/meili_data/dumps \
  -e MEILI_SNAPSHOT_INTERVAL_SEC=3600 \
  -v $(pwd)/meili_data:/meili_data \
  getmeili/meilisearch:v1.10
```

Latihan

Latihan 1: Setup & Index

Setup Meilisearch pake Docker. Buat index `movies` dengan primary key `id`. Add 10 dokumen film (title, director, year, genre, rating). Cek task status sampe succeeded.

Latihan 2: Filterable & Faceted Search

Dari data film di atas, config searchable attributes (title, director, genre), filterable attributes (year, genre, rating). Search "action" dengan filter `year >= 2020 AND genre = "Action"`. Tampilkan facet distribution by genre.

Latihan 3: Typo Tolerance

Cari film "interstallar" (sengaja typo). Catat hasilnya. Terus ubah typo tolerance settings — disable typo buat field `title`. Cek apakah hasil berubah. Atur minWordSizeForTypos biar kata < 5 char gak kena typo.

Latihan 4: Ranking Rules Integration

Buat endpoint Express GET `/api/search?q=...&category=...&minPrice=...&sort=...`. Integrasiin Meilisearch. Implementasi custom ranking `popularity:desc`. Return response: `{ hits, totalHits, facetDistribution, processingTimeMs }`.

03. Client-Side Search

Kapan Pake Client-Side Search?

Server-side search (PostgreSQL FTS / Meilisearch) biasanya pilihan utama. Tapi ada situasi client-side search lebih cocok:

Skenario	Server-Side	Client-Side
Dataset kecil (< 10k items)	□ Overkill	□ Simple
Data statis / jarang berubah	□	□ Satu kali fetch
Pencarian real-time	□	□ Kalau data besar
Autocomplete cepet	□ Kalo dedicated	□ Instant
Typo tolerance kompleks	□	□ Terbatas
Filter + facet	□	□ Manual JS
User offline mode	□ Butuh network	□ Bisa offline

Rule of thumb: kalau dataset < 5.000 item dan gak real-time sync, client-side search lebih simpel.

Fuse.js

Fuse.js adalah lightweight fuzzy-search library (gak punya dependency). Cocok buat client-side search di browser.

Setup

```
npm install fuse.js
```

CDN:

```
<script src="https://cdn.jsdelivr.net/npm/fuse.js@7.0.0/dist/fuse.basic.min.js">
```

Basic Usage

```
import Fuse from 'fuse.js';

const items = [
  { id: 1, title: 'Kopi Robusta Aceh', category: 'Minuman', price: 45000 },
  { id: 2, title: 'Teh Hijau Jawa Barat', category: 'Minuman', price: 35000 },
  { id: 3, title: 'Keripik Singkong Pedas', category: 'Makanan', price: 15000 }
];

const fuse = new Fuse(items, {
  keys: ['title', 'category'],
  threshold: 0.4   // 0 = exact, 1 = semua match
});

const results = fuse.search('kopi');
console.log(results);
// [
//   { item: { id: 1, ... }, refIndex: 0, score: 0.081 },
```

```
//   { item: { id: 2, ... }, refIndex: 1, score: 0.4  }  -- "Teh" match "kopi"?
// ]
```

Fuse.js Configuration

```
const fuse = new Fuse(items, {
  // --- Keys ---
  keys: ['title', 'description', 'tags'],

  // --- Fuzzy options ---
  threshold: 0.4,         // 0.0 = exact, 1.0 = match everything
  distance: 100,          // max distance for fuzzy matching (in characters)
  location: 0,            // where search starts in string
  findAllMatches: false,  // find all matches or just enough
  minMatchCharLength: 2,  // minimum length of match

  // --- Scoring ---
  useExtendedSearch: false, // enable AND/OR/NOT syntax
  ignoreLocation: false,    // ignore position in string
  ignoreFieldNorm: false,   // ignore field length norm

  // --- Threshold ---
  isCaseSensitive: false,
  shouldSort: true,
  includeScore: true,
  includeMatches: true,    // include match positions for highlighting

  // --- Weight ---
  // Array: { name: 'title', weight: 2 } — title 2x penting
  keys: [
    { name: 'title', weight: 2 },
    { name: 'description', weight: 1 },
    { name: 'tags', weight: 1.5 }
  ]
});
```

Threshold Explained

Threshold	Behavior	Use Case
0.0	Exact match only	Code / SKU search
0.2	Very strict, minor typo	Product codes
0.4	Moderate fuzzy	General search (default)
0.6	Loose fuzzy	Search suggestion
0.8-1.0	Almost everything match	Autocomplete broad

Extended Search

```
const fuse = new Fuse(items, {
  keys: ['title'],
  useExtendedSearch: true
});

// AND
fuse.search("'kopi 'robusta");   // must have kopi AND robusta

// OR
fuse.search("kopi | teh");       // kopi OR teh

// NOT
fuse.search("!kopi");            // exclude kopi

// Exact phrase
fuse.search('"kopi robusta"');   // exact phrase match
```

Weighted Search

```
const fuse = new Fuse(items, {
  keys: [
    { name: 'title', weight: 3 },       // title 3x lebih penting
    { name: 'tags', weight: 2 },
    { name: 'description', weight: 0.5 }
  ],
  threshold: 0.3
});
```

lunr.js

Alternatif client-side search. Punya index building mirip Elasticsearch — lebih cocok buat dataset lumayan besar.

Setup

```
<script src="https://unpkg.com/lunr@2.3.9/lunr.min.js"></script>
```

```
npm install lunr
```

Basic Usage

```
import lunr from 'lunr';

// 1. Build index
```

```
const idx = lunr(function() {
  this.ref('id');
  this.field('title');
  this.field('description');
  this.field('tags');

  this.add({ id: 1, title: 'Kopi Robusta Aceh',
             description: 'Kopi dengan cita rasa earthy',
             tags: 'kopi robusta aceh' });
  this.add({ id: 2, title: 'Teh Hijau Jawa Barat',
             description: 'Teh premium dari Gambung',
             tags: 'teh hijau jawa' });
  this.add({ id: 3, title: 'Keripik Singkong Pedas',
             description: 'Camilan pedas renyah',
             tags: 'keripik singkong pedas' });
});

// 2. Search
const results = idx.search('kopi');
// [
//   { ref: '1', score: 1.386, matchData: { ... } },
//   { ref: '2', score: 0.446, ... }
// ]
```

Perbandingan Fuse.js vs lunr.js

Aspek	Fuse.js	lunr.js
Algorithm	Bitap (fuzzy)	TF-IDF + inverted index
Fuzzy search	□ Built-in	□ Harus exact / wildcard
Index speed	O(n) linear	Build index dulu
Search speed	O(n)	O(1) lookup
Dataset besar	Lambat (>10k)	Cepet
Bundle size	~15KB	~30KB
Offline	□	□

Pilih Fuse.js — dataset kecil, butuh fuzzy, instant setup. **Pilih lunr.js** — dataset menengah (>5k), search cepet, butuh scoring.

Search dengan Debounce

Jangan panggil API tiap user ngetik — nanti banjir request. Debounce nunggu user berhenti ngetik.

```html
<html>
<head>
  <title>Search Demo</title>
  <style>
    .search-container { max-width: 600px; margin: 40px auto; font-family: sans-s
    #search-input { width: 100%; padding: 12px; font-size: 16px; border: 2px sol
    #search-results { margin-top: 12px; }
    .search-item { padding: 12px; border-bottom: 1px solid #eee; }
    .search-item h3 { margin: 0 0 4px 0; }
    .highlight { background: #fef08a; font-weight: 600; padding: 0 2px; border-r
    .no-results { color: #666; text-align: center; padding: 20px; }
    .loading { color: #666; text-align: center; padding: 10px; }
    #autocomplete-list { position: absolute; width: 100%; background: white; bor
                         border-top: none; max-height: 200px; overflow-y: auto;
    .autocomplete-item { padding: 10px 12px; cursor: pointer; }
    .autocomplete-item:hover { background: #f0f0f0; }
  </style>
</head>
<body>
<div class="search-container">
  <h1>Search Products</h1>
  <div style="position: relative;">
    <input type="text" id="search-input" placeholder="Cari produk..." autocomple
    <div id="autocomplete-list"></div>
  </div>
  <div id="search-results"></div>
</div>

<script src="https://cdn.jsdelivr.net/npm/fuse.js@7.0.0/dist/fuse.basic.min.js">
<script>
// ---------- Debounce ----------
function debounce(fn, delay = 300) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

// ---------- API Search ----------
async function searchProducts(query) {
  if (query.length < 2) {
    document.getElementById('search-results').innerHTML = '';
    document.getElementById('autocomplete-list').style.display = 'none';
    return;
  }
```

```
  const resultsEl = document.getElementById('search-results');
  resultsEl.innerHTML = '<div class="loading">🔍 Mencari...</div>';

  try {
    const res = await fetch(`/api/search?q=${encodeURIComponent(query)}&limit=10
    const data = await res.json();
    renderResults(data.hits, query);
    renderAutocomplete(data.hits, query);
  } catch (err) {
    resultsEl.innerHTML = '<div class="no-results">Gagal mencari. Coba lagi.</di
  }
}

// Debounced version
const debouncedSearch = debounce(searchProducts, 300);

// ---------- Event Listener ----------
document.getElementById('search-input').addEventListener('input', (e) => {
  debouncedSearch(e.target.value);
});

document.getElementById('search-input').addEventListener('keydown', (e) => {
  if (e.key === 'Escape') {
    document.getElementById('autocomplete-list').style.display = 'none';
  }
});

// Close autocomplete on click outside
document.addEventListener('click', (e) => {
  if (!e.target.closest('.search-container')) {
    document.getElementById('autocomplete-list').style.display = 'none';
  }
});
</script>
</body>
</html>
```

Autocomplete Dropdown

```
function renderAutocomplete(hits, query) {
  const list = document.getElementById('autocomplete-list');

  if (!hits || hits.length === 0 || query.length < 2) {
    list.style.display = 'none';
```

```
    return;
  }

  const suggestions = hits.slice(0, 5);
  list.innerHTML = suggestions.map((hit, i) => {
    const title = highlightText(hit.name || hit.title, query);
    return `<div class="autocomplete-item"
                 onclick="selectSuggestion('${hit.name || hit.title}')"
                 data-index="${i}">
              ${title}
            </div>`;
  }).join('');

  list.style.display = 'block';
}

function selectSuggestion(value) {
  document.getElementById('search-input').value = value;
  document.getElementById('autocomplete-list').style.display = 'none';
  searchProducts(value); // trigger full search
}

// Keyboard navigation for autocomplete
document.getElementById('search-input').addEventListener('keydown', (e) => {
  const list = document.getElementById('autocomplete-list');
  const items = list.querySelectorAll('.autocomplete-item');
  if (items.length === 0) return;

  let selectedIndex = -1;

  if (e.key === 'ArrowDown') {
    e.preventDefault();
    // find currently selected
    items.forEach((item, i) => {
      if (item.style.background === 'rgb(240, 240, 240)') selectedIndex = i;
    });
    selectedIndex = Math.min(selectedIndex + 1, items.length - 1);
    updateSelection(items, selectedIndex);
  } else if (e.key === 'ArrowUp') {
    e.preventDefault();
    items.forEach((item, i) => {
      if (item.style.background === 'rgb(240, 240, 240)') selectedIndex = i;
    });
    selectedIndex = Math.max(selectedIndex - 1, 0);
    updateSelection(items, selectedIndex);
  } else if (e.key === 'Enter' && selectedIndex >= 0) {
```

```
    e.preventDefault();
    items[selectedIndex].click();
  }
});

function updateSelection(items, index) {
  items.forEach((item, i) => {
    item.style.background = i === index ? '#f0f0f0' : 'white';
  });
}
```

Search Result Highlighting

```
function highlightText(text, query) {
  if (!text || !query) return text || '';

  // Escape regex special chars
  const escaped = query.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');

  // Split query into words, highlight each
  const words = escaped.split(/\s+/).filter(Boolean);
  let result = String(text);

  for (const word of words) {
    const regex = new RegExp(`(${word})`, 'gi');
    result = result.replace(regex, '<span class="highlight">$1</span>');
  }

  return result;
}

function renderResults(hits, query) {
  const resultsEl = document.getElementById('search-results');

  if (!hits || hits.length === 0) {
    resultsEl.innerHTML = '<div class="no-results">□ Tidak ada hasil untuk "' +
    return;
  }

  resultsEl.innerHTML = hits.map(hit => `
    <div class="search-item">
      <h3>${highlightText(hit.name || hit.title, query)}</h3>
      <p>${highlightText(hit.description || hit.body?.substring(0, 150), query)}
      <small>Kategori: ${hit.category} | Harga: Rp ${(hit.price || 0).toLocaleSt
    </div>
```

```
  `).join('');
}
```

Lazy Load Search Index

Buat dataset besar, jangan load index di awal. Fetch pas user siap search.

```
let fuseInstance = null;
let loading = false;

async function getFuse() {
  if (fuseInstance) return fuseInstance;
  if (loading) return null; // still loading

  loading = true;

  try {
    // Fetch index data from server (JSON)
    const res = await fetch('/api/search-index');
    const items = await res.json();

    fuseInstance = new Fuse(items, {
      keys: ['title', 'description', 'tags'],
      threshold: 0.4,
      includeScore: true,
      includeMatches: true
    });

    loading = false;
    return fuseInstance;
  } catch (err) {
    loading = false;
    throw err;
  }
}

async function searchLocal(query) {
  const fuse = await getFuse();
  if (!fuse) return [];

  const results = fuse.search(query);
  return results.slice(0, 20).map(r => ({
    ...r.item,
    score: r.score,
    matches: r.matches
```

```
  }));
}
```

Optimasi: Compress Index JSON

```
// Server: generate pre-built index
// /api/search-index → return array of { id, title, description, tags, category,

// Browser: cache with localStorage
async function getSearchIndex() {
  const CACHE_KEY = 'app_search_index';
  const CACHE_DURATION = 5 * 60 * 1000; // 5 menit

  const cached = localStorage.getItem(CACHE_KEY);
  if (cached) {
    const { data, timestamp } = JSON.parse(cached);
    if (Date.now() - timestamp < CACHE_DURATION) {
      return data;
    }
  }

  const res = await fetch('/api/search-index');
  const data = await res.json();

  localStorage.setItem(CACHE_KEY, JSON.stringify({
    data,
    timestamp: Date.now()
  }));

  return data;
}
```

Full Demo — Client-Side Search dengan Fuse.js

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fuse.js Search Demo</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: system-ui, sans-serif; background: #f8fafc; padding: 20p
    .container { max-width: 700px; margin: 0 auto; }
    h1 { margin-bottom: 20px; color: #1e293b; }
```

```
    .search-wrapper { position: relative; margin-bottom: 24px; }
    #search-input {
      width: 100%; padding: 14px 18px; font-size: 16px;
      border: 2px solid #e2e8f0; border-radius: 12px;
      outline: none; transition: border-color 0.2s;
    }
    #search-input:focus { border-color: #3b82f6; }

    .autocomplete-list {
      position: absolute; top: 100%; left: 0; right: 0;
      background: white; border: 1px solid #e2e8f0;
      border-top: none; border-radius: 0 0 12px 12px;
      max-height: 240px; overflow-y: auto; display: none;
      z-index: 100; box-shadow: 0 4px 12px rgba(0,0,0,0.1);
    }
    .autocomplete-item {
      padding: 10px 18px; cursor: pointer; font-size: 14px;
      border-bottom: 1px solid #f1f5f9;
    }
    .autocomplete-item:last-child { border-bottom: none; }
    .autocomplete-item:hover, .autocomplete-item.active { background: #eff6ff; }
    .autocomplete-item .highlight { color: #2563eb; font-weight: 600; }

    #search-meta { font-size: 13px; color: #64748b; margin-bottom: 12px; }
    .search-hit {
      background: white; border-radius: 10px; padding: 16px 20px;
      margin-bottom: 10px; box-shadow: 0 1px 3px rgba(0,0,0,0.06);
    }
    .search-hit h3 { margin-bottom: 4px; color: #1e293b; }
    .search-hit p { font-size: 14px; color: #475569; margin-bottom: 6px; line-he
    .search-hit .meta { font-size: 12px; color: #94a3b8; }
    .highlight { background: #fef08a; font-weight: 600; padding: 0 2px; border-r
    .no-results { text-align: center; padding: 40px; color: #94a3b8; }
    .loading { text-align: center; padding: 20px; color: #64748b; }
  </style>
</head>
<body>
<div class="container">
  <h1>□ Cari Produk</h1>

  <div class="search-wrapper">
    <input type="text" id="search-input" placeholder="Ketik nama produk..." auto
    <div class="autocomplete-list" id="autocomplete-list"></div>
  </div>

  <div id="search-meta"></div>
```

```
  <div id="search-results"></div>
</div>

<script src="https://cdn.jsdelivr.net/npm/fuse.js@7.0.0/dist/fuse.basic.min.js">
<script>
// ===== Data =====
const DATA = [
  { id: 1, name: 'Kopi Robusta Aceh', desc: 'Kopi robusta asli dataran tinggi Ga
  { id: 2, name: 'Teh Hijau Jawa Barat', desc: 'Teh hijau premium dari perkebuna
  { id: 3, name: 'Kopi Arabika Toraja', desc: 'Kopi arabika specialty dari Toraj
  { id: 4, name: 'Keripik Singkong Pedas', desc: 'Keripik singkong homemade deng
  { id: 5, name: 'Madu Hutan Sumbawa', desc: 'Madu hutan murni langsung dari Sum
  { id: 6, name: 'Keripik Pisang Coklat', desc: 'Keripik pisang balut coklat Bel
  { id: 7, name: 'Kopi Liberika Riau', desc: 'Kopi liberika langka asal Riau. Ci
  { id: 8, name: 'Teh Melati Nusantara', desc: 'Teh melati harum dengan campuran
  { id: 9, name: 'Abon Sapi Super', desc: 'Abon sapi asli 100% tanpa campuran. G
  { id: 10, name: 'Coklat Papua Asli', desc: 'Coklat murni dari biji kakao Papua
];

// ===== Fuse.js Init =====
const fuse = new Fuse(DATA, {
  keys: [
    { name: 'name', weight: 3 },
    { name: 'desc', weight: 1 },
    { name: 'cat', weight: 1 }
  ],
  threshold: 0.4,
  distance: 100,
  includeScore: true,
  includeMatches: true,
  minMatchCharLength: 2
});

// ===== Helpers =====
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

function escapeRegex(str) {
  return str.replace(/[.*+?^${}()|[\]\\]/g, '\\$&');
}
```

```
function highlight(text, query) {
  if (!text || !query) return text || '';
  const words = query.trim().split(/\s+/).filter(Boolean);
  let result = String(text);
  for (const word of words) {
    result = result.replace(new RegExp(`(${escapeRegex(word)})`, 'gi'), '<span c
  }
  return result;
}

// ===== Render =====
function renderAutocomplete(results, query) {
  const list = document.getElementById('autocomplete-list');
  if (!results.length || query.length < 2) {
    list.style.display = 'none';
    return;
  }
  const top = results.slice(0, 5);
  list.innerHTML = top.map(r =>
    `<div class="autocomplete-item">${highlight(r.item.name, query)}</div>`
  ).join('');
  list.style.display = 'block';

  // Click handler
  list.querySelectorAll('.autocomplete-item').forEach((el, i) => {
    el.addEventListener('click', () => {
      document.getElementById('search-input').value = top[i].item.name;
      list.style.display = 'none';
      doSearch(top[i].item.name);
    });
  });
}

function renderResults(results, query) {
  const resultsEl = document.getElementById('search-results');
  const metaEl = document.getElementById('search-meta');

  if (!results.length) {
    metaEl.textContent = '';
    resultsEl.innerHTML = `<div class="no-results">□ Tidak ada hasil untuk "<str
    return;
  }

  metaEl.textContent = `${results.length} hasil untuk "${query}"`;
  resultsEl.innerHTML = results.map(r => `
    <div class="search-hit">
```

```
      <h3>${highlight(r.item.name, query)}</h3>
      <p>${highlight(r.item.desc, query)}</p>
      <div class="meta">${r.item.cat} · Rp ${r.item.price.toLocaleString()} · Sk
    </div>
  `).join('');
}

function doSearch(query) {
  if (query.trim().length < 2) {
    document.getElementById('search-results').innerHTML = '';
    document.getElementById('search-meta').textContent = '';
    return;
  }
  const results = fuse.search(query.trim());
  renderResults(results, query.trim());
}

const debouncedSearch = debounce(doSearch, 250);
const debouncedAutocomplete = debounce(renderAutocomplete, 150);

// ===== Event =====
document.getElementById('search-input').addEventListener('input', (e) => {
  const q = e.target.value;
  if (q.length >= 2) {
    debouncedSearch(q);
    const results = fuse.search(q);
    debouncedAutocomplete(results.slice(0, 5), q);
  } else {
    document.getElementById('search-results').innerHTML = '';
    document.getElementById('search-meta').textContent = '';
    document.getElementById('autocomplete-list').style.display = 'none';
  }
});

document.getElementById('search-input').addEventListener('focus', (e) => {
  if (e.target.value.length >= 2) {
    document.getElementById('autocomplete-list').style.display = 'block';
  }
});

document.addEventListener('click', (e) => {
  if (!e.target.closest('.search-wrapper')) {
    document.getElementById('autocomplete-list').style.display = 'none';
  }
});
```

```
document.getElementById('search-input').addEventListener('keydown', (e) => {
  if (e.key === 'Escape') {
    document.getElementById('autocomplete-list').style.display = 'none';
  }
});
</script>
</body>
</html>
```

Latihan

Latihan 1: Fuse.js Setup

Ambil data 20 produk (buat dummy JSON). Setup Fuse.js dengan keys: name (weight 2), description (weight 1), tags (weight 1.5). Threshold 0.3. Uji search “kopi”, “teh manis”, “keripik”. Buat tabel perbandingan hasil.

Latihan 2: Debounce + Autocomplete

Buat HTML page dengan search input. Implement debounce 300ms. Tampilkan autocomplete dropdown 5 saran teratas pas user ngetik. Navigasi pake arrow keys + enter. Highlight text match di dropdown.

Latihan 3: Search Result Highlighting

Dari hasil search Fuse.js, tampilkan 10 hasil dengan: - Judul di-highlight (span.highlight) - Deskripsi snippet 100 karakter + highlight - Score relevansi (1 - score) ditampilkan - Kalo gak ada hasil, tampilkan “Tidak ada hasil untuk [query]”

Latihan 4: Lazy Load + lunr.js

Implementasi client-side search pake lunr.js. Data 100 item di-fetch dari API `/api/products-index`. Lazy load: index di-build pas pertama kali user search. Cache index di localStorage (5 menit). Bandingkan performance Fuse.js vs lunr.js buat 100 item.

04. Search Architecture

Single vs Dedicated Search Service

Single Database Search (PostgreSQL FTS)

Paling simpel — search langsung di database utama.

```
graph LR
    User-->API
    API-->PostgreSQL[(PostgreSQL<br/>+ GIN Index)]
    PostgreSQL-->API
    API-->User
```

Cocok buat: - Dataset < 1 juta row - Search bukan fitur utama - Tim kecil / MVP - Infra minimal

Kelebihan: gak ada service baru, data selalu konsisten (read your own writes), gak perlu sync. **Kekurangan:** performance turun di scale, gak ada typo tolerance, query complex jadi lambat.

Dedicated Search Engine (Meilisearch / Elasticsearch)

Search engine terpisah — performa optimal.

```
graph LR
    User-->API
    API-->PostgreSQL[(PostgreSQL<br/>Primary DB)]
    API-->Meilisearch[(Meilisearch<br/>Search Engine)]
    PostgreSQL-- Sync -->Meilisearch
    Meilisearch-->API
    API-->User
```

Cocok buat: - Dataset besar (> 1M docs) - Search adalah primary feature - Butuh typo tolerance, filter, faceted - Butuh response < 50ms

Kelebihan: performa tinggi, fitur search lengkap, scalable. **Kekurangan:** tambah infra, data bisa stale (tergantung sync), maintenance cost.

Decision Matrix

Kriteria	PostgreSQL FTS	Dedicated Search Engine (Meilisearch)
Setup	Built-in	Docker / binary
Consistency	Strong (same DB)	Eventual (sync delay)
Search speed	50-200ms	10-50ms
Typo tolerance	Manual	Built-in
Ranking	ts_rank	Ranking rules + custom
Faceted search	Manual GROUP BY	Built-in
Scaling read	Replica	Horizontal shard
DevOps overhead	None	Sedang
Use case	Secondary / hybrid	Primary / high perf

Data Sync Strategy

Data di PostgreSQL harus sync ke search engine. Tiga strategi:

1. Webhook / Application-Level Sync

Waktu write ke DB, langsung kirim ke search engine.

```
sequenceDiagram
    participant User
    participant API
    participant DB
    participant Queue
    participant Worker
    participant Search

    User->>API: POST /products
    API->>DB: INSERT product
    DB-->>API: success
    API->>Queue: ENQUEUE index.product
    API-->>User: 201 Created
    Worker->>Queue: DEQUEUE index.product
    Queue-->>Worker: product data
    Worker->>Search: addDocuments(product)
    Search-->>Worker: taskUid
    Worker->>Worker: log success
```

Pros: real-time (detik), kontrol penuh, bisa retry logic. **Cons:** kalo service crash di tengah, data gak ke-index. Butuh queue (BullMQ / Redis) biar reliable.

```
// Product service
async function createProduct(req, res) {
  const product = await db.products.create(req.body);

  // Queue indexing (async)
  await indexQueue.add('index-product', {
    action: 'upsert',
    data: product
  });

  res.status(201).json(product);
}

// Worker
indexQueue.process(async (job) => {
  const { action, data } = job.data;
```

```
  if (action === 'upsert') {
    await searchClient.index('products').addDocuments([data]);
  } else if (action === 'delete') {
    await searchClient.index('products').deleteDocument(data.id);
  }
});
```

2. Cron-Based Sync

Berkala (tiap 5-30 menit) sync semua data yang berubah.

```
// Script: scripts/sync-search.js
const cron = require('node-cron');
const { db } = require('./db');
const { searchClient } = require('./search');

// Sync tiap 15 menit
cron.schedule('*/15 * * * *', async () => {
  const lastSync = await getLastSyncTime();

  // Ambil data yang berubah sejak lastSync
  const products = await db.products.findMany({
    where: { updatedAt: { gt: lastSync } }
  });

  if (products.length === 0) return;

  // Update ke Meilisearch
  await searchClient.index('products').addDocuments(products);

  // Update lastSync
  await setLastSyncTime(new Date());
});
```

Pros: sederhana, gak butuh queue. **Cons:** delay sampai 15 menit, data bisa stale, full scan query tiap cron.

3. CDC — Change Data Capture

Debezium / pglogical — capture perubahan dari PostgreSQL WAL (Write-Ahead Log).

```
graph LR
    User-->API
    API-->PostgreSQL
    PostgreSQL-->WAL[(WAL Log)]
```

```
    WAL-->Debezium[Debezium<br/>CDC Connector]
    Debezium-->Kafka[Kafka Topic<br/>db.products.cdc]
    Kafka-->Consumer[CDC Consumer]
    Consumer-->Search[Meilisearch]
```

Pros: real-time, gak perlu modifikasi aplikasi, capture semua perubahan (termasuk direct SQL). **Cons:** kompleks, butuh Kafka, infrastruktur berat. Overkill buat startup.

Comparison

Strategy	Delay	Complexity	Reliability	Use Case
Webhook	Detik	Medium	Retry via queue	Most apps
Cron	5-30 menit	Low	Bisa skip	Batch / gak real-time
CDC	Real-time	High	WAL-level	Enterprise / high volume

Hybrid Search

Kombinasi PostgreSQL FTS + Meilisearch — ambil kelebihan masing-masing.

```
graph TD
    User-->Frontend[Frontend]
    Frontend-->HybridAPI[Hybrid Search API]
    HybridAPI-->Meilisearch[Meilisearch<br/>Primary Search]
    HybridAPI-->PostgreSQL[PostgreSQL<br/>Fallback + Metadata]
    Meilisearch-- hits -->HybridAPI
    PostgreSQL-- enriched data -->HybridAPI
    HybridAPI-->User

    subgraph Sync
        PostgreSQL-- webhook -->Meilisearch
    end
```

Kapan pake hybrid?

- **Primary search → Meilisearch**: autocomplete, fuzzy search, filter, faceted
- **Detail data → PostgreSQL**: ambil data lengkap pas user klik hasil search
- **Fallback → PostgreSQL FTS**: kalau Meilisearch down, masih bisa search basic

Implementasi

```
async function hybridSearch(req, res) {
  const { q, filter, page = 1, limit = 20 } = req.query;

  try {
    // 1. Primary — Meilisearch
    const meiliResults = await searchClient.index('products').search(q, {
      filter: buildFilter(filter),
      limit,
      offset: (page - 1) * limit
    });

    // 2. Enrich — ambil data lengkap dari PostgreSQL
    const ids = meiliResults.hits.map(h => h.id);
    const products = await db.products.findMany({
      where: { id: { in: ids } },
      select: {
        id: true,
        name: true,
        description: true,
        category: true,
        price: true,
        stock: true,
        images: true,
        rating: true,
        reviews: true
      }
    });

    // Map back preserving Meilisearch order
    const productMap = new Map(products.map(p => [p.id, p]));
    const enriched = meiliResults.hits.map(h => ({
      ...productMap.get(h.id),
      _score: h._score,
      _rankingScore: h._rankingScore
    }));

    res.json({
      hits: enriched,
      totalHits: meiliResults.totalHits,
      facetDistribution: meiliResults.facetDistribution
    });

  } catch (err) {
    console.error('Meilisearch failed, fallback to PostgreSQL FTS:', err.message
```

```
    // 3. Fallback — PostgreSQL FTS
    const pgResults = await db.$queryRaw`
      WITH query AS (
        SELECT plainto_tsquery('indonesian', ${q}) AS q
      )
      SELECT
        p.*,
        ts_rank_cd(p.search_vector, q.q) AS score
      FROM products p, query q
      WHERE p.search_vector @@ q.q
        AND p.deleted = false
      ORDER BY score DESC
      LIMIT ${limit} OFFSET ${(page - 1) * limit}
    `;

    res.json({
      hits: pgResults,
      totalHits: pgResults.length,
      fallback: true
    });
  }
}
```

Search Analytics

Pantau apa yang user cari — buat improve search relevansi.

Popular Searches

```
CREATE TABLE search_logs (
  id SERIAL PRIMARY KEY,
  query TEXT NOT NULL,
  results_count INTEGER DEFAULT 0,
  user_id INTEGER,
  ip_address INET,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_search_logs_query ON search_logs(query);
CREATE INDEX idx_search_logs_created ON search_logs(created_at);
```

```
// Log setiap search
async function logSearch(query, resultsCount, userId, ip) {
  await db.searchLogs.create({
    data: {
```

```
      query: query.toLowerCase().trim(),
      resultsCount,
      userId,
      ipAddress: ip
    }
  });
}

// Get popular searches (hari ini)
const popular = await db.$queryRaw`
  SELECT
    query,
    COUNT(*) as frequency,
    AVG(results_count) as avg_results
  FROM search_logs
  WHERE created_at >= CURRENT_DATE
  GROUP BY query
  ORDER BY frequency DESC
  LIMIT 20
`;
```

Zero Results Tracking

Query yang gak dapet hasil = sinyal buat improve.

```
// Track zero-result queries
async function trackZeroResults() {
  const zeroResults = await db.$queryRaw`
    SELECT query, COUNT(*) as frequency
    FROM search_logs
    WHERE results_count = 0
    GROUP BY query
    ORDER BY frequency DESC
    LIMIT 20
  `;

  // Kirim notifikasi tim product
  console.log('□ Zero result queries:', zeroResults);

  return zeroResults;
}

// Auto-suggest buat zero results
function suggestForZeroResults(query) {
  // Pake Fuse.js buat cari kata terdekat
  const fuse = new Fuse(knownGoodQueries, { threshold: 0.5 });
```

```
  const suggestions = fuse.search(query);

  return suggestions.slice(0, 3).map(s => s.item);
}
```

Analytics Dashboard

```
// Endpoint ringkasan analytics
app.get('/api/admin/search-analytics', async (req, res) => {
  const { startDate, endDate } = req.query;

  const [popular, zeroResults, trends] = await Promise.all([
    // Top queries
    db.$queryRaw`
      SELECT query, COUNT(*) as freq, AVG(results_count) as avg_res
      FROM search_logs
      WHERE created_at BETWEEN ${startDate} AND ${endDate}
      GROUP BY query ORDER BY freq DESC LIMIT 20
    `,

    // Zero results
    db.$queryRaw`
      SELECT query, COUNT(*) as freq
      FROM search_logs
      WHERE created_at BETWEEN ${startDate} AND ${endDate}
        AND results_count = 0
      GROUP BY query ORDER BY freq DESC LIMIT 20
    `,

    // Daily trend
    db.$queryRaw`
      SELECT DATE(created_at) as date, COUNT(*) as total_searches,
             COUNT(*) FILTER (WHERE results_count > 0) as successful_searches
      FROM search_logs
      WHERE created_at BETWEEN ${startDate} AND ${endDate}
      GROUP BY DATE(created_at) ORDER BY date
    `
  ]);

  res.json({ popular, zeroResults, trends });
});
```

Multi-Language Search

Dataset multilingual butuh strategi khusus.

Approaches

Approach	Pros	Cons	Use Case
Single index, auto-detect	Simple, satu endpoint	Stemming cuma buat satu bahasa	Mayoritas satu bahasa
Per-language field	Stemming sesuai tiap bahasa	Ukuran index lebih besar	Konten bilingual
Per-language index	Isolasi sempurna	Query routing complex	Multi-region app
No stemming (simple)	Konsisten semua bahasa	Gak dapet stemming benefit	Short text / names

Per-language Field (PostgreSQL)

```
ALTER TABLE documents ADD COLUMN search_vector_en tsvector;
ALTER TABLE documents ADD COLUMN search_vector_id tsvector;

CREATE FUNCTION docs_search_update() RETURNS trigger AS $$
BEGIN
  NEW.search_vector_en := to_tsvector('english', coalesce(NEW.title,'') || ' ' |
  NEW.search_vector_id := to_tsvector('indonesian', coalesce(NEW.title,'') || '
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Search: coba English dulu, fallback Indonesian
SELECT * FROM documents
WHERE search_vector_en @@ plainto_tsquery('english', 'economic growth')
   OR search_vector_id @@ plainto_tsquery('indonesian', 'economic growth')
ORDER BY
  CASE
    WHEN search_vector_en @@ plainto_tsquery('english', 'economic growth') THEN
    ELSE 2
  END;
```

Per-language Field (Meilisearch)

```
// Index dengan field per bahasa
const docs = [
```

```
  {
    id: 1,
    title_en: 'Coffee Production in Indonesia',
    title_id: 'Produksi Kopi di Indonesia',
    desc_en: 'Indonesian coffee production reached record high',
    desc_id: 'Produksi kopi Indonesia mencapai rekor tertinggi',
  }
];

await searchClient.index('documents').addDocuments(docs);

// Search — cari di dua field
await searchClient.index('documents').search('produksi kopi', {
  attributesToSearchOn: ['title_id', 'desc_id']
});

await searchClient.index('documents').search('coffee production', {
  attributesToSearchOn: ['title_en', 'desc_en']
});
```

Language Detection

```
// Simple detection pake karakter
function detectLanguage(query) {
  // Indonesian common words
  const idWords = ['dan', 'di', 'ke', 'dari', 'yang', 'dengan', 'untuk', 'pada',
  // English common words
  const enWords = ['and', 'the', 'of', 'to', 'in', 'for', 'with', 'is', 'this',

  const words = query.toLowerCase().split(/\s+/);
  let idScore = words.filter(w => idWords.includes(w)).length;
  let enScore = words.filter(w => enWords.includes(w)).length;

  if (idScore > enScore) return 'id';
  if (enScore > idScore) return 'en';

  // Fallback: karakter spesifik
  if (/[^a-zA-Z0-9\s]/.test(query)) return 'id'; // ada karakter non-ASCII
  return 'en'; // default
}

// Gunakan di search
async function searchMultiLang(query) {
  const lang = detectLanguage(query);
  const fields = lang === 'id'
```

```
    ? ['title_id', 'desc_id']
    : ['title_en', 'desc_en'];

  return await searchClient.index('documents').search(query, {
    attributesToSearchOn: fields
  });
}
```

Full Production Architecture

```
graph TB
    subgraph Client
        Browser[Web App]
        Mobile[Mobile App]
    end

    subgraph API Layer
        LB[Load Balancer]
        API[Express API]
        WS[WebSocket<br/>Auto-update]
    end

    subgraph Database
        PG[(PostgreSQL<br/>Primary DB)]
        PGReplica[(PostgreSQL<br/>Read Replica)]
    end

    subgraph Search
        Meili[Meilisearch<br/>Primary]
        MeiliReplica[Meilisearch<br/>Replica]
    end

    subgraph Sync
        Queue[(Redis Queue<br/>BullMQ)]
        Worker[Sync Worker]
    end

    subgraph Analytics
        ELK[(ELK / Grafana)]
        SearchLogs[Search Logs]
    end

    Client-->LB
    LB-->API
```

```
API-->PG
API-->Meili

PG-- CDC / Webhook -->Queue
Queue-->Worker
Worker-->Meili

Meili-->MeiliReplica

API-->SearchLogs
SearchLogs-->ELK

WS-- real-time index -->Client
```

Production Checklist

1. **Separate search service** — jangan gabung di API server
2. **Async sync** — queue-based, jangan blocking request
3. **Circuit breaker** — fallback ke PostgreSQL kalau Meilisearch down
4. **Monitoring** — track search latency, error rate, popular queries
5. **A/B testing** — test ranking changes sama user
6. **Backup** — dump Meilisearch index berkala
7. **Staging index** — jangan langsung nge-index ke production
8. **Rate limiting** — proteksi dari abuse

Latihan

Latihan 1: Data Sync Webhook

Buat Express endpoint POST `/api/products`. Setelah insert ke PostgreSQL, kirim data ke Meilisearch via BullMQ queue. Guarantee: kalo queue gagal, data tetap masuk ke DB. Implement retry 3x.

Latihan 2: Hybrid Search API

Buat endpoint GET `/api/search?q=...` yang: - Primary search pake Meilisearch - Enrich data pake PostgreSQL (ambil semua field termasuk image_url, rating, stock) - Fallback ke PostgreSQL FTS kalau Meilisearch error - Response: `{ hits: [...], totalHits, source: 'meilisearch'|'postgres' }`

Latihan 3: Search Analytics

Buat middleware search logger yang nyimpen query, results_count, user_id, ip, created_at ke tabel `search_logs`. Implement endpoint GET `/api/admin/search-analytics` yang return: - Top 10 popular queries hari ini - Zero result queries (results_count = 0) — top 5 - Daily search trend 7 hari terakhir

Latihan 4: Multi-Language Search

Buat tabel `documents_multilang` dengan kolom `title_en`, `title_id`, `content_en`, `content_id`, `lang`. Implementasi search endpoint yang: - Detect language dari query - Search di field sesuai bahasa - Fallback ke field lain kalau hasil sedikit (< 3) - Tampilkan label “Bahasa Indonesia” / “English” di response